

Retro-Wave-Game - Master Blueprint Plan

Revision: v0.2 **Date:** 2026-10-03
Primary visual target: Neon Hollow concept - B / First Person

1. Vision

Retro-Wave-Game is a real-time horror game with a retro-wave/synthwave visual identity, but the target is more specific than a generic neon game.

The visual target is the supplied **Neon Hollow - B / First Person** concept:

- Dark luxury / decayed hotel
- Illustrated / comic-book influenced 3D
- Cyan / teal lighting
- Hot magenta lighting
- Deep blue / black shadows
- Neon emissive fixtures
- Wet reflective floors
- Strong reflections
- Controlled bloom
- Atmospheric depth / fog
- Cinematic first-person horror

The underlying technology should be efficient and playable. Visual effects should create the style without making the game unnecessarily expensive to run.

2. Production principle

Build a **small, polished vertical slice first**.

Do not build the entire world before proving that the first-person corridor can deliver: - the required look - the required controls - the required interaction - the required performance - the required atmosphere

The vertical slice becomes the technical and visual benchmark for later expansion.

3. Team production model

Account	Responsibility	Main output
Account 1	Photo -> Game Tool / Asset Production	Clean reusable 3D assets
Account 2	Gemini Research / Asset Library / Downloads & Uploads	Approved assets, research and rights records
Account 3	ChatGPT Game Builder / Integration	Playable game, systems, rendering and testing

Handoff flow



4. Visual specification

4.1 Palette

Primary visual relationship:

cyan / teal + hot magenta + deep blue / black

Accent colours must remain controlled so that neon lighting remains visually meaningful.

4.2 Lighting

The lighting system should support: - Cyan / teal practical lights - Hot magenta practical lights - Deep controlled shadows - Emissive materials - Local accent lights - Strong contrast

4.3 Materials

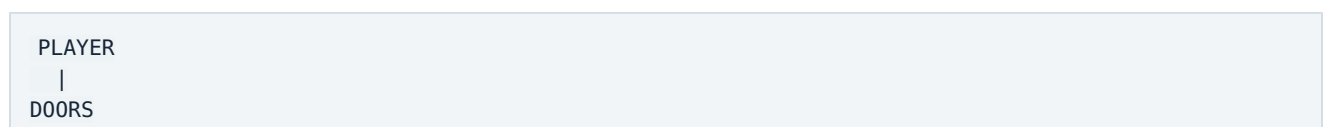
Priority material families: - Wet / reflective floor - Worn wood - Painted wall surfaces - Marble / tile - Metal - Glass - Carpet / fabric

4.4 Atmosphere

Target: - Controlled bloom - Atmospheric depth - Fog / haze where useful - Reflections - Cinematic contrast - Stylised / comic treatment

4.5 Composition

Corridors should create a strong visual vanishing point.



```
|  
LIGHTS  
|  
REFLECTIONS  
|  
ARCHWAY  
|  
MAGENTA / DARKNESS
```

5. First-person player

Primary mode: **first person**.

Core stack:

```
PLAYER  
|  
CAMERA  
|  
HANDS  
|  
WEAPON / EQUIPMENT  
|  
FLASHLIGHT
```

Exact movement values remain OPEN until tested in the vertical slice.

6. First vertical slice

The first playable slice should contain: - First-person player - One hotel corridor - 5-10 doors - Neon lighting - Wet reflective floor - Reflections - One room branching from the corridor - One enemy - Flashlight - Weapon / equipment - Basic interaction - Basic UI - Ambient hotel / horror audio

Slice success condition

A player can load in, move through the corridor, interact with the environment, experience the target visual treatment, encounter the enemy system and complete a small playable loop.

7. Initial asset budget

Start with approximately **25-40 reusable assets**, not hundreds of unique models.

Architecture

```
corridor_wall, corridor_floor, corridor_ceiling, archway, door_frame, door,  
corner_piece, skirting, ceiling_panel
```

Hotel props

```
luggage_cart, suitcase, table, chair, lamp, plant, painting, mirror, telephone,  
small_table, cabinet
```

Interactables

key, door_lock, switch, light_switch, fuse_box, note, pickup_object, weapon, flashlight

Characters

player, player_hands, enemy, enemy_variant

Effects

rain, mist, dust, spark, flickering_light, dirt/blood decal, water

8. Account 1 specification - Photo -> Game

Mission

Turn approved photographs, references and visual material into clean, reusable, game-ready assets for Account 3.

Required pipeline

```
PHOTO / REFERENCE
-> OBJECT / SHAPE INTERPRETATION
-> 3D GEOMETRY
-> MATERIALS
-> UVs
-> OPTIMISATION
-> GLB / GLTF
-> GAME-READY ASSET
```

Asset requirements

Models should have, where appropriate: - Correct scale - Correct orientation - Clean geometry - Sensible polygon density - Collision support - LOD

Textures should use the maps actually needed by the material: - Base colour / albedo - Normal - Roughness - Metalness - AO - Emissive

9. Account 2 specification - Research / Asset Library

Mission

Find useful material, verify rights, catalogue it and hand clean information/assets to Account 3.

Research scope

- Textures
- Materials
- 3D models

- Animations
- Audio
- VFX
- Reference images
- Code examples
- Open-source libraries
- Shader examples
- Lighting and post-processing research
- Performance techniques
- Useful development tools

Rights rule

Free to download does not automatically mean commercially usable.

Every external asset must record:

```
asset_name, source, creator, licence, commercial_use, modification_allowed,  
attribution_required, download_date, source_url, notes
```

10. Account 3 specification - Game Builder

Mission

Turn the approved assets and technical research into the actual playable game.

Owns

- Game architecture
- Scene structure
- Player controller
- Camera
- Interaction
- Collision
- Lighting
- Materials
- Rendering
- VFX
- Enemy systems
- UI
- Audio integration
- Game states
- Performance
- Testing
- Asset integration

Engineering rule

Do not stop the entire build because a final asset is missing. Use a correctly sized placeholder and replace it later.

11. Performance strategy

This is a tight-budget project.

Prefer: - Reusable modular assets - Instancing - Sensible polygon counts - Efficient texture sizes - LOD where useful - Controlled post-processing - Efficient lighting - Streaming/loading where appropriate - Mobile-aware fallbacks where required

Measure expensive features before keeping them.

12. Development stages

Stage 0 - Documentation foundation

Set up the blueprint, handovers, benchmark rules and decision log.

Stage 1 - Technical foundation

Build the basic application, loading, scene, first-person camera, input and test environment.

Stage 2 - Corridor prototype

Build corridor geometry, player movement, collision and basic interaction.

Stage 3 - Visual lock

Implement Neon Hollow lighting, wet floor, reflections, emissive fixtures, atmosphere and stylised treatment.

Stage 4 - Playable corridor slice

Add room, equipment, interaction, UI and audio.

Stage 5 - Enemy encounter

Add the enemy prototype and encounter loop.

Stage 6 - Vertical slice benchmark

Polish, test, capture evidence and create a benchmark.

Stage 7 - World expansion

Only after the vertical slice is stable:

Lobby -> Corridors -> Rooms -> Stairs -> Service Areas -> Basement -> Special
Nightmare Areas

13. Benchmark system

A **BENCHMARK** is a confirmed working point.

A benchmark must contain: - Benchmark ID - Human-readable name - Git commit / tag - Blueprint revision - Build reference - Visual tests passed - Gameplay tests passed - Known accepted issues - Rollback purpose - Date

Rules

1. Never overwrite a confirmed benchmark.
2. Never call an untested build a benchmark.
3. New work starts forward from the current benchmark.
4. A broken later build can roll back to the latest appropriate benchmark.
5. A repaired state gets a new benchmark.

14. Planned benchmark ladder

Benchmark	Target
BENCHMARK-000	Repository/application foundation loads
BENCHMARK-001	First-person corridor prototype works
BENCHMARK-002	Neon Hollow visual lock demonstrated
BENCHMARK-003	Playable corridor slice
BENCHMARK-004	Enemy encounter slice
BENCHMARK-005	Vertical slice complete

These are planned IDs, not claimed completed benchmarks.

15. Open decisions - workshop order

DECISION 01 - Platform

Choose the first shipping target and any later expansion target.

Status: OPEN

DECISION 02 - Input

Lock keyboard/mouse, controller and mobile/touch scope.

Status: OPEN

DECISION 03 - Engine layer

Godot 4 with GDScript is locked by the user. The repo root contains project.godot; see docs/GODOT-STRUCTURE.md. Exact stable minor version and renderer/effect validation remain to be measured on target hardware.

Status: LOCKED — Godot 4 / GDScript

DECISION 04 - Player movement

Lock movement speeds, sprint, crouch, jump/lean rules and interaction range.

Status: OPEN

DECISION 05 - Weapon/equipment

Lock what appears in the first slice.

Status: OPEN

DECISION 06 - Enemy

Lock appearance, detection, movement and encounter behaviour.

Status: OPEN

DECISION 07 - Hotel/world logic

Lock fixed versus modular/procedural room assembly.

Status: OPEN

DECISION 08 - Commercial asset policy

Lock approved sources, licences and acceptable AI asset tiers.

Status: OPEN

DECISION 09 - Performance target

Lock minimum hardware and practical budgets.

Status: OPEN

DECISION 10 - Audio

Lock original versus sourced audio strategy.

Status: OPEN

DECISION 11 - UI

Lock HUD and interaction-prompt structure.

Status: OPEN

16. Change control

Every blueprint revision must record: - Version - Date - What changed - Why - Affected accounts - Whether a new benchmark is required

A blueprint change does not automatically mean the game is broken. The benchmark system separates **design evolution** from **working build state**.

17. Current next step

Import the Godot foundation, verify its main scene on the development PC, then establish the first-person corridor prototype. Resolve the remaining OPEN decisions in Section 15 as needed for that slice.

Lock only what needs to be fixed for the next build.

Do not over-design future systems before the corridor slice is proven.

Change log

v0.1 - 2026-10-03

- Created living blueprint structure.
- Locked Neon Hollow B / First Person as the primary visual target.
- Established three-account production flow.
- Established vertical-slice-first strategy.
- Established benchmark rollback process.
- Recorded the initial open decision list.

v0.2 - 2026-10-03

- Locked Godot 4 and GDScript following user confirmation.
- Established Godot project/feature/resource folders and selected asset integration.
- All three production accounts must target Godot-compatible handoffs.
- No new benchmark: structure was checked, editor/runtime execution remains unverified.